

M08 Teaching Book

Beginner-first, self-contained Files, APIs & Data Ingestion. External media helps visualization; the local labs and this book remain sufficient when a video changes or disappears.

VIDEO FIRST -> close video -> SOLO TEACHING BOOK -> guided reproduction -> independent task -> validate -> explain in own words -> protected Review only after genuine saved attempt -> fresh changed retry if weak -> evidence/mastery

Core engineering rule: source truth and run evidence come before convenience. Never hide dropped rows, skipped pages, failed retries, cursor changes, schema drift, late data, or overwritten raw inputs.

DE08-01 - Ingestion architecture: source -> landing -> staging

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

Whole workshop architecture pass: APIs/files/databases -> normalization -> warehouse; ignore platform-specific install steps.

Starting state / prerequisite

M07 frozen; M04-M06 Python/file/Git basics available.

Outcome

Design a reproducible source-to-landing-to-staging boundary and state what is immutable, validated and transformed at each layer.

Teach it from zero

Ingestion is a chain of responsibility. Source is what the producer exposes. Landing preserves raw evidence as received. Staging parses and validates it into a shape safe for later transformation. Do not overwrite raw bytes just because a later schema is easier to query. A trustworthy pipeline can show what arrived, what was rejected, and what changed afterward.

Teacher demonstration

Create folders source/, landing/, staging/, quarantine/. Copy one batch file into landing without renaming its bytes; record size + SHA-256; parse it into staging only after validation.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Design a 4-layer folder/contract for a weather-station file feed. State exactly what can change at each boundary and what evidence must never be overwritten.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain ingestion architecture: source -> landing -> staging in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[dlt REST API tutorial](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-02 - Defensive CSV ingestion

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

File-ingestion portions only; course lab owns malformed-row/quarantine rules.

Starting state / prerequisite

DE08-01 mastered.

Outcome

Read CSV defensively with explicit encoding/header/delimiter/schema expectations, quarantine malformed business rows and reconcile counts.

Teach it from zero

CSV is deceptively simple: delimiters, quotes, newlines, encodings, headers, missing fields and business-invalid values can all break assumptions. Defensive ingestion separates parsing validity from business validity, counts every source business row, and quarantines bad rows with a reason instead of silently dropping them.

Teacher demonstration

Use `csv.DictReader`. Require the expected header, parse `ship_date` as a real date and `weight_kg` as numeric. Send bad rows to quarantine and prove `seen = accepted + quarantined`.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal `ParcelFlow` fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Write a CSV reader for a changed shipment file with one invalid date and one missing required ID. Quarantine errors with reason and prove count reconciliation.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain defensive csv ingestion in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Python 3.14 csv](#)

[pandas read_csv](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-03 - JSON and nested-record ingestion

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)
API/nested-data normalization portions; course lab owns exact flattening grain.

Starting state / prerequisite

DE08-02 mastered.

Outcome

Parse nested JSON without silently dropping arrays/objects, flatten only the fields required by the target grain and preserve raw payload evidence.

Teach it from zero

JSON can represent nested objects and arrays, so flattening changes shape. Decide the target grain first. Preserve the raw payload, then extract only needed paths. An event array normally means one output row per event, not one row per whole document. Missing nested keys need an explicit policy.

Teacher demonstration

Flatten normal/input/tracking_events.json to one row per tracking event with event_id, shipment_id, region, scan_type, event_time and device_id. Keep the original JSON untouched.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Flatten a changed nested JSON payload whose root contains devices[] and each device contains readings[]. State output grain before code.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain json and nested-record ingestion in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Python 3.14 json](#)

[pandas read_json](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-04 - Parquet ingestion and metadata inspection

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

File/warehouse portions as visual context only; Apache Parquet docs are format authority.

Starting state / prerequisite

DE08-03 mastered.

Outcome

Explain columnar Parquet layout, verify file identity/magic/footer metadata, and inspect schema/row-group metadata with a compatible reader when available.

Teach it from zero

Parquet is a column-oriented binary format. A file contains column chunks organized in row groups plus footer metadata; readers use metadata to locate needed columns. The format begins and ends with PAR1 magic. Metadata inspection is not the same as opening a text file. A compatible Parquet engine such as PyArrow exposes schema and row-group details.

Teacher demonstration

Run `m08_parquet_probe.py` on the supplied real Apache Parquet fixture. Prove magic at both ends and footer metadata length. If PyArrow is installed, also record row count, row groups and schema; otherwise record the dependency honestly.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Inspect the supplied Parquet file. Without a reader, prove binary identity/footer safely; with PyArrow, record schema/row groups/rows. Explain what you could not verify if PyArrow is unavailable.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain parquet ingestion and metadata inspection in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Apache Parquet overview](#)

[Apache Parquet metadata](#)

[pandas read_parquet](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-05 - REST API contracts, status codes and timeouts

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

API extraction portions; Requests docs + localhost lab define timeout/status contract.

Starting state / prerequisite

DE08-04 mastered.

Outcome

Call an HTTP API with explicit timeout/status handling and prove that JSON decoding alone is not success evidence.

Teach it from zero

HTTP success is a contract, not "I got JSON". A server can return JSON error details with a 4xx/5xx status. Set a timeout, check status before trusting payload, separate connection/timeouts from HTTP errors, and keep enough request metadata to reproduce a failure without logging secrets.

Teacher demonstration

Call the localhost /health and /records endpoints. Prove /records without auth is 401. Call /slow with a short timeout and save the timeout evidence.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Write a function that accepts URL + timeout, rejects non-2xx responses before trusting JSON, and distinguishes HTTP error from timeout/connection error.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain rest api contracts, status codes and timeouts in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Requests Quickstart - status/timeouts](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-06 - API authentication, pagination and rate limits

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)
API extraction/pagination concepts; course localhost API proves token/pagination/rate-limit behavior.

Starting state / prerequisite

DE08-05 mastered.

Outcome

Keep credentials out of code, traverse pagination exactly once, obey rate-limit signals and reconcile fetched record counts.

Teach it from zero

Credentials belong in environment/configuration boundaries, not committed source. Pagination means one logical dataset spans several responses. Follow the API next-page contract until it ends, protect against loops, and reconcile fetched IDs/counts. Rate limits are part of the source contract; respect Retry-After or documented quota behavior.

Teacher demonstration

Fetch all localhost records with Bearer token m08-demo-token. The data spans three logical pages. Save IDs and prove no duplicate page/record traversal.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Build a paginator that stops only when next_page is null, refuses a repeated page token, keeps token outside source code and reconciles unique IDs.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain api authentication, pagination and rate limits in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Requests authentication](#)

[dlt REST API source pagination](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-07 - Retries, backoff, jitter and transient failures

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

Reliability/failure-handling concepts; urllib3 docs + lab define bounded retry behavior.

Starting state / prerequisite

DE08-06 mastered.

Outcome

Retry only appropriate transient/idempotent operations with bounded attempts, exponential backoff/jitter and observable retry evidence.

Teach it from zero

Retries help only when failure may be transient. Bound attempts. Retry appropriate idempotent requests/statuses, not every error. Exponential backoff reduces repeated pressure; jitter prevents many clients from retrying in lockstep. Every retry should be observable so hidden instability is not mistaken for a clean run.

Teacher demonstration

Use the guided client against the localhost server. Page 2 rate-limits once and page 3 returns one temporary 503. Prove the client eventually retrieves all records and logs exactly which attempts were retried.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Implement bounded retry for GET on 429/503 only. Record attempt number/status/backoff. Explain why blindly retrying POST can be unsafe.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain retries, backoff, jitter and transient failures in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[urllib3 Retry](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-08 - Database extraction and safe query boundaries

VIDEO FIRST route

[DataTalksClub - Docker/PostgreSQL data-engineering workshop](#)

Use only Python/database ingestion/query-boundary portions. DO NOT follow Docker setup in M08; Docker is M10.

Starting state / prerequisite

DE08-07 mastered; M07 database modeling complete.

Outcome

Extract from a database with parameterized read queries, explicit columns, deterministic bounds and no accidental source mutation.

Teach it from zero

Database extraction should be boring and safe: explicit columns, read-only SELECT logic, parameters instead of string-built values, deterministic ordering/bounds, and a known extraction grain. Large production extracts often require chunking/snapshots/replica strategies, but the learner must first prove boundary correctness.

Teacher demonstration

Query source_extract.db for orders updated from 10:00 inclusive to 12:30 exclusive using parameter placeholders. Save row count and SQL text. Do not UPDATE/DELETE the source.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Parameterize a time-window SELECT against the SQLite source. Return explicit columns and deterministic order; save query + parameters + count.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain database extraction and safe query boundaries in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Python 3.14 sqlite3](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-09 - Historical batch landing and raw metadata

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

Source/file ingestion and durable loading mental model; course lab defines immutable landing metadata.

Starting state / prerequisite

DE08-08 mastered.

Outcome

Land historical files without overwriting raw bytes; capture source name, size, hash, ingest time and batch/business date metadata.

Teach it from zero

Historical backfills are easiest to trust when landing preserves each source object exactly once and metadata explains identity. Useful metadata includes original name/path, size, content hash, ingest time, business/batch date and source system. Landing is evidence, not a convenient scratch folder.

Teacher demonstration

Land the three batch files to a clean landing directory. Record SHA-256 and business-row count for each. Rerun landing and prove existing bytes are unchanged.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Land three changed batch files under a run/date prefix. Save name, bytes, SHA-256, business rows and ingest timestamp; rerun without overwriting changed bytes.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain historical batch landing and raw metadata in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Python 3.14 pathlib](#)

[Python 3.14 hashlib](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-10 - Incremental loads and high-water marks

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

Incremental/state concepts; current dlt incremental docs define cursor/state model.

Starting state / prerequisite

DE08-09 mastered.

Outcome

Load only new/changed rows using a stable cursor/high-water rule that does not lose ties at the boundary.

Teach it from zero

A high-water mark tells the next run where to continue. Timestamp-only cursors can lose rows when several records share the same timestamp. A composite cursor such as (updated_at, record_id) gives a deterministic total order. State should move only after the destination commit succeeds.

Teacher demonstration

Process incremental_source.csv in two runs: first five rows, then the remainder. Use (updated_at, record_id) as cursor. Prove I006 is not lost even though I004-I006 share 10:10.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Process the 8-row incremental fixture using composite cursor. Stop first run after five rows, then finish; prove tie-boundary row I006 is not lost and third run returns zero.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain incremental loads and high-water marks in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[dlt incremental loading](#)

[dlt cursor incremental](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-11 - CDC concepts and change-event semantics

VIDEO FIRST route

[Binod Suman Academy - Change Data Capture with Debezium/Kafka](#)

What/why/how CDC + Debezium/Kafka demo; course book controls generic event semantics.

Starting state / prerequisite

DE08-10 mastered.

Outcome

Interpret insert/update/delete change events, before/after images and source ordering metadata without pretending a batch poll is CDC.

Teach it from zero

CDC captures changes from a transaction/change log and emits operation semantics such as create/update/delete, often with before/after state and source ordering metadata. Polling a table by updated_at is incremental extraction, not true CDC. Downstream consumers must define delete handling, ordering and replay behavior.

Teacher demonstration

Apply cdc_events.jsonl in seq order to an in-memory current-state map. Prove A1 is updated, A2 is deleted and A3 exists. Explain why the event log remains valuable even after current state is reconstructed.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Consume the supplied CDC log to derive current state plus an audit trail. State delete policy and what sequence field protects.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain cdc concepts and change-event semantics in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Debezium architecture](#)

[Debezium event flattening](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-12 - Schema drift, compatibility, time zones and quarantine

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)
Schema normalization/change validation concepts; course lab adds UTC/quarantine contracts.

Starting state / prerequisite

DE08-11 mastered.

Outcome

Detect compatible/incompatible drift, normalize timestamps to UTC, quarantine invalid rows and avoid silently coercing bad values.

Teach it from zero

Schema drift is any change to expected fields/types/meaning. Some changes are additive/compatible; others break consumers. Timestamp strings also carry semantics: normalize instants to UTC while preserving enough raw evidence to diagnose source issues. Invalid rows go to quarantine with reason; do not replace bad numeric/time values with zero.

Teacher demonstration

Process `drift_events.csv`. Parse amount + event_time; normalize valid times to UTC. Quarantine bad numeric and bad timestamp rows. Reconcile all six source rows.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal `ParcelFlow` fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Write a contract validator that accepts extra optional field `source_note`, but quarantines invalid amount/time. Normalize valid timestamps to UTC and preserve raw row.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain schema drift, compatibility, time zones and quarantine in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Python 3.14 datetime/timezone](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-13 - Checkpoint/state management and replay safety

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

Stateful/incremental concepts; dlt state docs are current reference.

Starting state / prerequisite

DE08-12 mastered.

Outcome

Persist checkpoint state only after successful durable output and prove a crash/replay cannot skip uncommitted data.

Teach it from zero

Checkpoint state is part of correctness. If you advance a cursor before output is durably committed, a crash can make the next run skip data forever. Correct order is: read from old state -> write/validate -> commit output -> atomically persist new state. Replay from an old state must be safe.

Teacher demonstration

Simulate destination failure after candidate I006 is read but before commit. Prove stored state remains I005; rerun should read I006 again rather than skip it.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Write pseudocode or Python showing state is saved only after destination success. Inject a failure before commit and prove state did not advance.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain checkpoint/state management and replay safety in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[dlt advanced state management](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-14 - Idempotent landing and content identity

VIDEO FIRST route

[PyCon US 2025 - Building Resilient Data Pipelines: The Power of Idempotency](#)

Whole focused talk; then implement course content-identity/idempotent landing task.

Starting state / prerequisite

DE08-13 mastered.

Outcome

Use stable business/content identity so rerunning the same input does not create duplicate output while changed content remains detectable.

Teach it from zero

Idempotency means repeating the same operation produces the same final state. For files, SHA-256 can be a strong content identity signal; business/event keys may be the dedup identity for records. Do not confuse filename with content identity: a renamed duplicate file can still be identical bytes.

Teacher demonstration

Hash all landed batch files, then run landing twice. Prove destination contains one copy per intended file/content and hashes stay unchanged.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Use content hash + event key to prevent duplicates. Test same filename/same bytes, renamed/same bytes and same filename/changed bytes; define each outcome.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain idempotent landing and content identity in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Python 3.14 hashlib](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-15 - Late-arriving data, event time and processing time

VIDEO FIRST route

[Confluent Developer - Event Time and Watermarks](#)

12-minute lesson: event vs processing time, out-of-order events, watermarks and late data.

Starting state / prerequisite

DE08-14 mastered.

Outcome

Distinguish event time from arrival/processing time, classify lateness against an explicit tolerance and explain watermark trade-offs.

Teach it from zero

Event time says when the source event happened; processing/arrival time says when the pipeline saw it. Late data arrives after the allowed delay. Watermarks are progress assumptions used by streaming systems; a longer allowance improves completeness but increases latency. M08 teaches the concept without requiring Flink runtime.

Teacher demonstration

For `late_events.json`, classify each event using 10-minute allowed lateness. Save IDs and delay seconds of late events. Explain why sorting only by `arrived_at` changes the business timeline.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal `ParcelFlow` fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Classify the six late-event fixtures at 10-minute tolerance. Recompute at 20 minutes and explain completeness-vs-latency trade-off.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain late-arriving data, event time and processing time in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Confluent Event-Time Processing](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-16 - Ingestion observability: counts, latency and run metadata

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

Validation/dashboard/run metadata concepts; course lab emits exact observability fields.

Starting state / prerequisite

DE08-15 mastered.

Outcome

Emit run-level evidence for rows seen/accepted/quarantined, retries, bytes, latency, checkpoint and status so failures can be diagnosed.

Teach it from zero

Observable ingestion emits evidence, not just "success". At minimum capture run_id/time, source, input rows/bytes, accepted/quarantine counts, retry count, elapsed/latency, checkpoint before/after, destination result and final status/error. Metrics must reconcile and should not leak credentials.

Teacher demonstration

Create one run record from the normal smoke job. Verify seen = accepted + quarantine where applicable and retries are visible rather than hidden.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

Emit a JSONL run record with counts, bytes, retries, elapsed_ms, cursor_before/after, status and quarantine path. Prove no token/password is logged.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain ingestion observability: counts, latency and run metadata in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[Python 3.14 logging](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.

DE08-17 - Ingestion integration, reconciliation and backfill proof

VIDEO FIRST route

[DataTalksClub DE Zoomcamp 2026 - From APIs to Warehouses: AI-Assisted Data Ingestion with dlt](#)

Whole-workflow review only; integrated build remains independent and local.

Starting state / prerequisite

DE08-01 through DE08-16 mastered.

Outcome

Run an end-to-end ingestion job, rerun it idempotently, backfill historical data and reconcile destination records to source/control totals.

Teach it from zero

Integration proves the pieces work together: inventory inputs -> land raw -> parse/validate -> extract/fetch -> deduplicate -> commit -> checkpoint -> observe -> reconcile. Then rerun unchanged input and perform a historical backfill. A backfill is not allowed to corrupt current state or duplicate already committed business events.

Teacher demonstration

Run m08_smoke_runner.py from a clean normal/output folder. Save the generated JSON evidence. Rerun it and confirm landing/idempotency controls remain stable. Then explain how you would add one older batch without resetting unrelated state.

Guided reproduction

Open the matching Guided Labs section. Reproduce the pattern once with the normal ParcelFlow fixtures while saving commands/output rather than transcribing the lesson.

Independent task

From a clean output folder run the full normal ingestion smoke, rerun unchanged inputs, then add one hypothetical older batch. Write a reconciliation table and backfill safety explanation.

Validate

Save the source/input identity, intended row/event grain, counts before/after, explicit error/quarantine evidence where relevant, and one independent reconciliation. A plausible output without these controls is not mastery.

Recover

Reduce the problem to the earliest broken boundary: source bytes/contract -> parse/fetch -> validation -> identity/dedup -> durable write -> checkpoint -> reconciliation. Repair one cause, then rerun from clean evidence rather than manually editing final output.

Explain back

Explain ingestion integration, reconciliation and backfill proof in 60-90 seconds: what problem it solves, one failure mode, and the evidence you would inspect before claiming success.

Current authoritative reference(s)

[dlt REST API tutorial](#)

[dlt incremental loading](#)

Master only when you can reproduce the skill on changed data without reopening Review. A percentage score never substitutes for required evidence.